Advanced Software Construction Techniques – Assignment 1

Group 4: Gonçalo Sampaio (202206636) and Rémi Linossier (202603466)

November 2026

1 Project Selection Problem (0/1 Knapsack)

The **0/1 Knapsack problem** consists of **selecting a subset of items** out of a given list inside a **knapsack with a fixed maximum weight**, so that the subset **maximizes the total value without exceeding the capacity** of the bag. Each item has a **fixed weight and value**, can only be **selected once** and **cannot be divided into a fraction** (e.g., selecting half an item is not allowed) [1].

In the context of **Project Selection**, the 0/1 Knapsack problem can be applied to selecting a subset of projects under a limited budget (the knapsack's maximum capacity), where each project has a cost (weight) and a benefit (value). The goal is to maximize the total benefit of the selection without exceeding the available budget.

2 Problem DSL Design

A **Domain Specific Language** was designed to allow users to model portfolio selection problems by specifying a set of available projects, their costs and benefits and a maximum budget that can be spent in the selection. The DSL was developed using **BESSER** where a UML Class Diagram was created to define its concepts, attributes, constraints and relationships.

Table 2 lists the Problem DSL concepts. The corresponding class diagram is shown in Figure 1.

- **SelectionProblem** represents the project selection problem (0/1 Knapsack) defined by a list of available projects and a budget, grouping all candidate projects and defining the budget that can be spent.
- **Project** represents a candidate project, identified by a unique ID and containing a cost and a benefit that represent, respectively, the weight and value of an item in the original problem.
- **SelectionResult** represents a feasible selection of projects for a specific SelectionProblem and includes its total cost and benefit. It was created to separate the problem definition from its possible results, allowing the problem to exist with zero or more feasible selections.

The class diagram was exported to B-UML Python code: `code/problem_DSL_class_diagram.py`.

2.1 Domain Rules

Several domain rules were introduced to ensure the consistency and validity of the problem instances and OCL constraints were used to enforce conditions that cannot be fully expressed through the metamodel's attributes and associations. These rules are detailed in Appendix A.

2.2 Example Object Model

Three test cases were selected to demonstrate the application of the DSL, as documented in `docs/Test-Cases.md` [2, 3, 4]. The first test case was used to create an Object Diagram in BESSER that represents a problem with five candidate projects and a budget of 26, where each project is represented as an instance of the Project class, associated with a single SelectionProblem instance [2] (Figure 2). The object diagram was exported to B-UML Python code: `code/problem_DSL_object_diagram.py`.

3 Solution DSL Design

A Domain Specific Language was designed to implement the mathematical optimisation equation. It represents a cost function thanks to binary variables, linear and quadratic coefficients. The DSL was developed using **BESSER** where a UML Class Diagram was created to define its concepts, attributes and relationships.

Table 3 describes the Solution DSL concepts, and Figure 3 contains the corresponding class diagram.

The class diagram was exported to B-UML Python code: `code/solution_DSL_class_diagram.py`.

- **QuboModel** represents a Quadratic Unconstrained Binary Optimization formulation, defined by the value of the constant. It groups all decision variables, linear and quadratic terms which create the cost equation.
- **BinaryVariable** represents a binary decision variable (0/1), identified by a name and representing either a project selection variable or a slack variable used to encode constraints (maximum size of the knapsack).
- **LinearTerm** represents first-order terms in the polynomial equation, defined by an integer coefficient associated with a single BinaryVariable to model the contribution of this specific project to the result.
- **QuadraticTerm** represents second-order terms in the polynomial equation, defined by an integer coefficient associated with two BinaryVariables to model the maximum budget constraint.

3.1 Domain Rules

Several domain rules were introduced to ensure the consistency and validity of the solution instances and OCL constraints were used to enforce conditions that cannot be fully expressed through the metamodel's attributes and associations. These rules are detailed in Appendix A.

3.2 Example Object Model

The first test case was also used to create the corresponding Solution Object Model, representing the Project Selection problem using the QUBO formulation. The model contains five binary decision variables (x_A to x_E), representing the five candidate projects, and five slack binary variables (s_1 to s_5), used to encode the budget constraint. The QuboModel groups these variables together with the linear and quadratic terms representing the mathematical cost function. The original optimization problem maximizes the total benefit of the selected projects while respecting a budget of 26. The intended QUBO transformation converts benefit maximization into minimization and incorporates the budget constraint through a quadratic penalty function [5]:

$$f(x,s) = -\sum_i b_i x_i + \lambda \left(\sum_i c_i x_i + s - B \right)^2 \qquad (1)$$

Here b_i and c_i represent each project's benefit and cost, respectively; λ is the penalty coefficient; and

$s = s_1 + 2s_2 + 4s_3 + 8s_4 + 16s_5$ encodes the unused budget. The model is illustrated in Figure 4 and the object diagram was implemented in B-UML Python code: `code/solution_DSL_object_diagram.py`.

The mapping between both DSLs is summarized in Table 4, while LLM assistance and validation are documented in Table 6.

4 T1 : Problem to Solution Model Transformation

Building upon the mathematical foundation established in Section 3.2, the T1 step implements an automated Model-to-Model (M2M) transformation. This script bridges the Problem DSL and the Solution DSL by executing the following operational sequence:

- **Dynamic Extraction & Validation:** The script dynamically parses the input Problem Object Model (e.g., `problem_DSL_object_diagram.py`) using Python's `runpy` module to extract the `SelectionProblem` and its associated `Project` instances. It performs internal validations to ensure structural integrity prior to transformation, such as verifying all the constraints defined in the Problem DSL.
- **Coefficient Computation:** Rather than relying on external symbolic math solvers, the algorithm directly factorizes the expanded QUBO objective function defined in Equation (1). It automatically calculates the optimal slack variable weights required to bound the budget space, and maps the project attributes (costs and benefits) into explicit constant, linear, and quadratic mathematical coefficients. [6] [7]
- **Solution Model Instantiation:** Finally, the algorithm instantiates the computed coefficients into their corresponding Solution DSL objects. It generates the root `QuboModel`, creates `BinaryVariable`s for both the decision and slack variables, and maps the calculated values into `LinearTerm` and `QuadraticTerm` relationships.

The output of the T1 transformation is an automatically generated B-UML Python script. This generated file contains a concrete Solution Object Model that fully encapsulates the problem's physical constraints and objectives, serving as the direct input second transformation.

5 T2 : Solution DSL to Quantum Model

The T2 phase is a Model-to-Model (M2M) transformation that maps the QUBO Solution ObjectModel into an abstract BESSER Quantum Model (`quantum_instance.py`).

The transformation operates through three main steps:

- **QUBO to Ising Mapping:** Binary variables $(0, 1)$ are mapped into Ising spin states (± 1) [6]. Linear terms are converted into single-qubit Z-rotation parameters, while quadratic terms map to two-qubit interaction parameters.
- **Quantum Metamodel Mapping:** The Ising formulation is converted into abstract BESSER quantum elements representing p layers of QAOA [8]:
  1. *Initial Superposition:* `HadamardGate` objects to initialize the quantum register.
  2. *Linear Rotations:* `RZGate` objects corresponding to linear QUBO terms.
  3. *Quadratic Interactions:* Abstract ZZ-coupling sequences combining `PauliXGate` (CNOT) and `RZGate` elements [9].
  4. *Mixer Layer:* `RXGate` objects for state transitions.
  5. *Measurement:* `Measurement` objects binding quantum register bits to classical register bits.
- **Solver Parameter Integration:** Algorithm settings such as QAOA depth p, variational angles (γ, β), and shot counts are read from `solver_config.json` and stored within the abstract model attributes.

The output of T2 is a BESSER Quantum ObjectModel (`quantum_instance.py`). This model acts as the input for Transformation 3.

6 T3 : Quantum Model to Qiskit Code

The final step of the pipeline (T3) is a Model-to-Text (M2T) transformation. It takes the abstract BESSER Quantum Model generated in T2 and translates it into an executable Python script using the Qiskit framework. Beyond simple circuit generation, T3 automatically integrates a classical validation layer to bridge the gap between raw quantum bitstrings and the original domain problem.

The algorithm executes the following sequence:

- **Traceability and Dynamic Extraction:** T3 guarantees end-to-end traceability by parsing metadata from the T2 and T1 outputs. It backtracks to the original Problem Object Model to dynamically extract the underlying business data (candidate projects, costs, benefits, budget, and slack weights) without relying on hardcoded instances.
- **Target Code Generation:** The script invokes the `besser.generators.qiskit.QiskitGenerator` to synthesize the baseline quantum simulation code. It then dynamically patches the execution parameters (such as the number of simulation `shots`) based on the `solver_config.json` configuration.
- **Domain Decoding and Validation:** Raw quantum outputs (probabilistic bitstrings) are meaningless without domain context. T3 automatically appends a post-processing analysis script to the generated Qiskit file. This injected logic performs several key tasks during execution:
  1. Decodes Qiskit's measurement counts back into domain selections (e.g., Projects A, B, E).
  2. Evaluates constraints, discarding invalid states that exceed the maximum budget.
  3. Calculates the total benefit of the measured feasible solutions.
  4. Computes a deterministic, brute-force classical baseline to establish the true optimal solution, allowing for the direct calculation of the quantum circuit's approximation ratio and empirical success probability.

The final output is an executable script (`qiskit_circuit.py`). When run, it builds the QAOA circuit, executes it on a local quantum simulator, and directly prints a comprehensive domain specific evaluation comparing the quantum measurements against the classical optimal baseline.

7 Validation and Limitations

The transformation pipeline was validated using three Knapsack instances with different sizes, costs and budgets (Small, Medium and Large). T1 was tested by verifying the generated QUBO coefficients and comparing the optimal solutions against exhaustive classical enumeration. T2 and T3 were validated by successfully generating and running Qiskit circuits using the Aer Simulator. Each circuit was executed with 1024 shots. The measured bitstrings were decoded into project selections, verified against the available budget, and compared with brute-force solutions. The results are summarized in Table 5. The best feasible selections achieved approximation ratios of 100% for all instances. However, QAOA used fixed parameters ($p = 1$) without angle optimization. Full QUBO feasibility remained low (1.46%, 0.49%, and 0.20% for Small, Medium, and Large, respectively). An optimal full QUBO state was measured only once, in the Small instance. These results demonstrate a functional transformation pipeline, but not a fully reliable quantum optimization problem solver.

References

[1] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Springer, 2004. https://doi.org/10.1007/978-3-540-24777-7.

[2] J. Burkardt, *KNAPSACK_01 — Data for the 01 Knapsack Problem*, datasets P02 and P04. https://people.sc.fsu.edu/~jburkardt/datasets/knapsack_01/knapsack_01.html.

[3] K. Lian, *Hands-on Optimization with OR-Tools in Python*, Chapter 8: The Knapsack Problem, 2023. https://kunlei.github.io/python-ortools-notes/ip-knapsack.html.

[4] Lolik-Bolik, *Knapsack_problem_0-1*, GitHub repository. https://github.com/Lolik-Bolik/Knapsack_problem_0-1.

[5] CodingThrust, "[Rule] Knapsack to QUBO," *problem-reductions*, GitHub Issue #116, 2026. https://github.com/CodingThrust/problem-reductions/issues/116.

[6] A. Lucas, *Ising formulations of many NP problems*. Frontiers in Physics, 2:5, 2014. https://doi.org/10.3389/fphy.2014.00005.

[7] F. Glover, G. Kochenberger, and Y. Du, *Quantum Bridge Analytics I: A Tutorial on Formulating and Using QUBO Models*, https://arxiv.org/abs/1811.11538.

[8] E. Farhi, J. Goldstone, and S. Gutmann, *A Quantum Approximate Optimization Algorithm*, https://arxiv.org/abs/1411.4028.

[9] A. Barenco, C. H. Bennett, R. Cleve, D. P. DiVincenzo, N. Margolus, P. Shor, T. Sleator, J. A. Smolin, and H. Weinfurter, *Elementary gates for quantum computation*. Physical Review A, 52(5):3457–3467, 1995, https://arxiv.org/abs/quant-ph/9503016.

A Domain Rules

DSL	Rule	Description
Problem	R1	The budget must be a non-negative integer.
Problem	R2	Each project's cost must be a non-negative integer.
Problem	R3	Each project's benefit must be a non-negative integer.
Problem	R4	Each Project must have a unique ID within its SelectionProblem.
Problem	R5	A SelectionProblem must contain at least one Project.
Problem	R6	Projects referenced by a SelectionResult must belong to the same SelectionProblem.
Problem	R7	The sum of selected project benefits must equal totalBenefit.
Problem	R8	The sum of selected project costs must equal totalCost.
Problem	R9	A SelectionResult's totalCost must not exceed the budget.
Solution	R1	Each BinaryVariable must have a unique name.
Solution	R2	Each LinearTerm coefficient must be non-zero.
Solution	R3	Each QuadraticTerm coefficient must be non-zero.
Solution	R4	Each QuadraticTerm must reference two different BinaryVariables.
Solution	R5	Each BinaryVariable must have a unique LinearTerm.
Solution	R6	Each pair of BinaryVariables must have a unique QuadraticTerm.
Solution	R7	LinearTerms must be factorized as much as possible.
Solution	R8	QuadraticTerms must be factorized as much as possible.

Table 1: Domain rules for the Problem and Solution DSLs.

Tables and Figures

Table 2: Problem DSL concept list.

Concept	Intrinsic Properties	Extrinsic Properties
SelectionProblem	budget: Integer	Contains at least one project and zero or more SelectionResults.
Project	id: String; cost: Integer; benefit: Integer	Belongs to one SelectionProblem and is a candidate to its solution.
SelectionResult	totalCost: Integer; totalBenefit: Integer	Belongs to one SelectionProblem and references zero or more Projects.

Table 3: Solution DSL concept list.

Concept	Intrinsic Properties	Extrinsic Properties
QuboModel	constant: Integer	Contains the BinaryVariables, LinearTerms and QuadraticTerms.
BinaryVariable	name: String	Belongs to one QuboModel.
LinearTerm	coefficient: Integer	Belongs to one BinaryVariable.
QuadraticTerm	coefficient: Integer	Belongs to two BinaryVariables.

Table 4: Mapping between Problem DSL and Solution DSL concepts.

Problem DSL	Solution DSL	Mapping
SelectionProblem	QuboModel	Represents the optimization problem through a mathematical cost function.
Project	BinaryVariable	A project corresponds to a binary variable indicating whether it is selected (1) or not (0).
Project.benefit	LinearTerm	Contributes to the objective function, with negative coefficients to represent benefit maximization as minimization.
Project.cost	LinearTerm, QuadraticTerm	Contributes to the penalty terms used to encode the budget constraint.
SelectionProblem.budget	QuboModel, BinaryVariable, LinearTerm, QuadraticTerm	Determines the penalty constant, coefficients and slack encoding.
SelectionResult	*No direct equivalent*	Represents a feasible project selection in the original domain, rather than an element of the QUBO formulation.

Table 5: QAOA validation results (1024 shots per instance).

Instance	Qubits	Optimum	Best Measured	Ratio	QUBO Feasible
Small	10	51	51	100%	1.46%
Medium	13	107	107	100%	0.49%
Large	19	431	431	100%	0.20%

Table 6: LLM usage log

Goal	Tool & Model	What Was Wrong and Validation
Finding real and verified test cases	ChatGPT	Compared the test cases against the original benchmark sources and checked the optimal solutions.
Evaluating the inclusion of SelectionResult in Problem DSL	ChatGPT	Reviewed the decision against the domain requirements and the metamodel structure.
Corrections in OCL syntax	Gemini & ChatGPT	Some answers from LLM were wrong because of the version of Besser. We used Quality Check of the class diagram in Besser to validate the modifications.
Creating the solution object code based on the solution class code. Creating the object by hand in the GUI of Besser would have been difficult because of the high number of links between objects.	Gemini	At first some links were missing an end because of ambiguous names. Then we manually stepped through the code to look at the object instances and links, plus Quality Check of the object diagram in Besser.
Creating the mathematical formulation of the solution because we haven't found a test case with	Gemini	Quality Check of the object diagram in Besser.
Improving grammar and writing and verifying spelling	ChatGPT	Manually reviewed suggested corrections to ensure that the content, terminology, and intended meaning remained unchanged.
Reviewing the Problem-to-QUBO transformation (T1)	Gemini & ChatGPT	Reviewed the transformation logic and QUBO coefficients and validated the generated models against all three test instances and their known optimal solutions.
Implementation and debugging support for transformation 2 and 3	Gemini & ChatGPT	Revised the QUBO-to-Ising conversion, quantum circuit generation and result decoding.
Automating the transformation pipeline and solver configuration	ChatGPT	Assisted with the pipeline script and JSON configuration integration.
Interpreting QAOA simulation results	ChatGPT	Reviewed the measurement statistics, feasibility rates and approximation ratios.

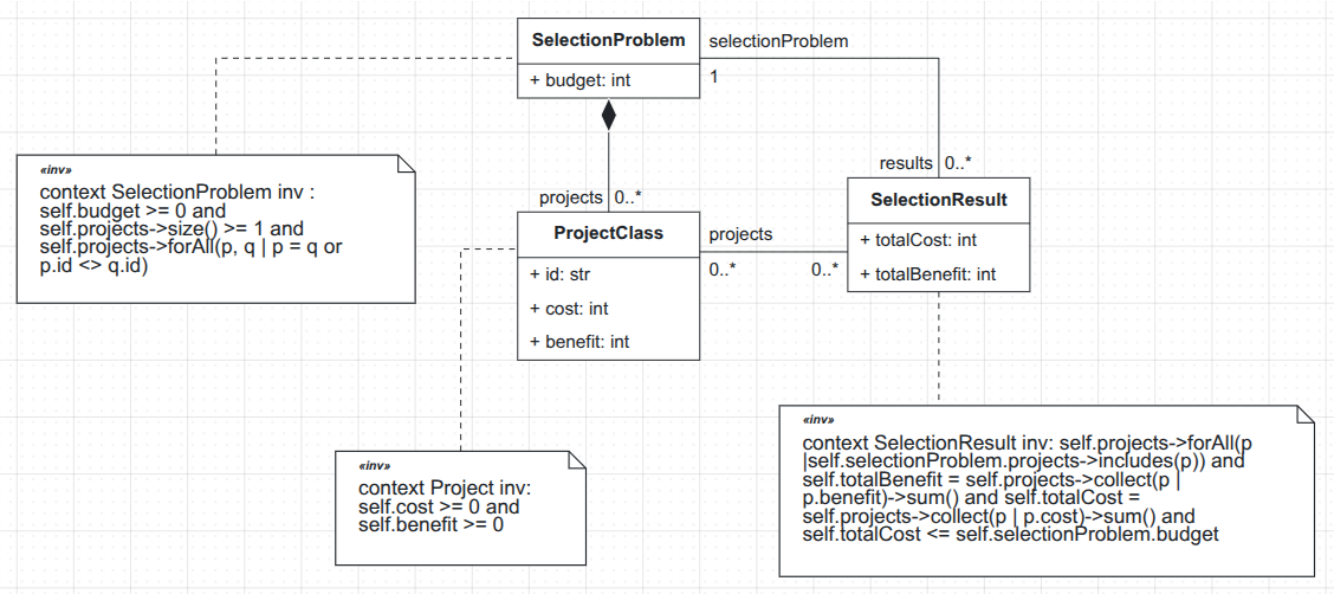


Figure 1: Problem DSL class diagram.

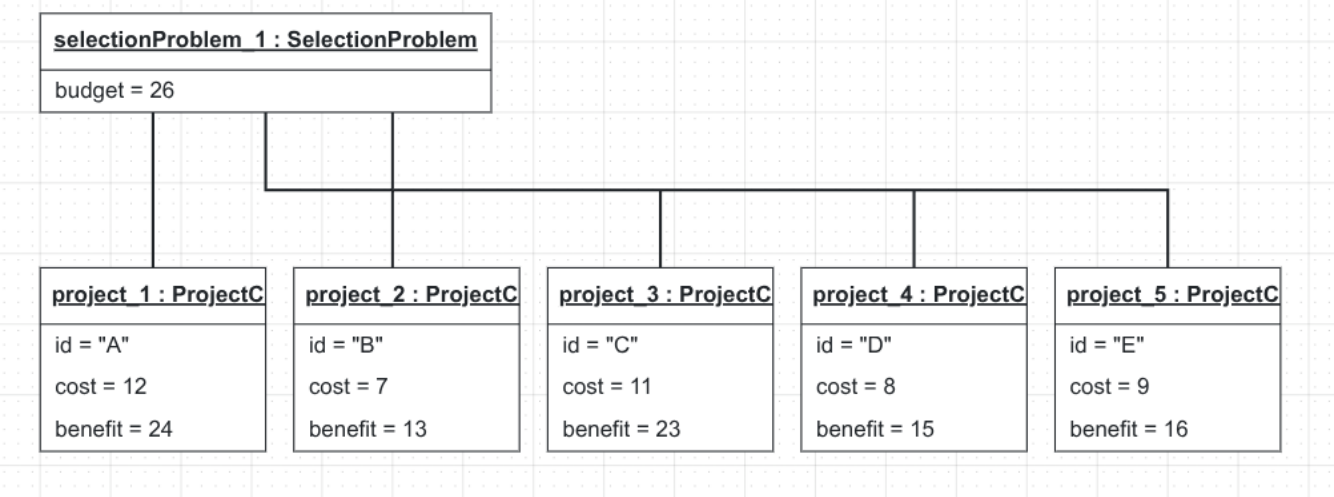


Figure 2: Problem DSL object diagram for Test Case 1.

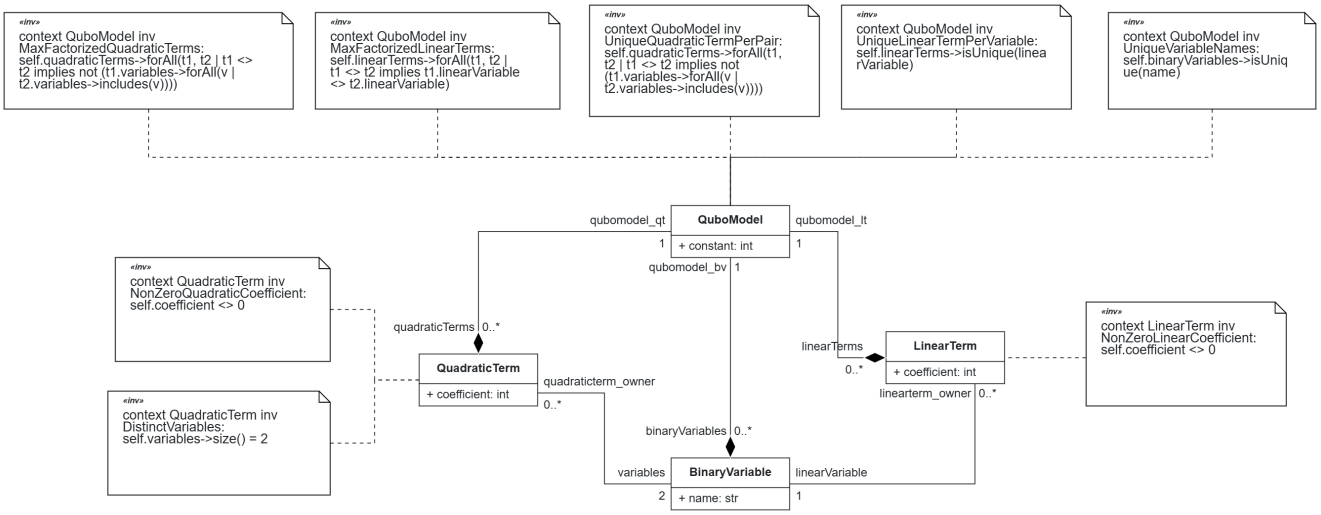


Figure 3: Solution DSL class diagram.

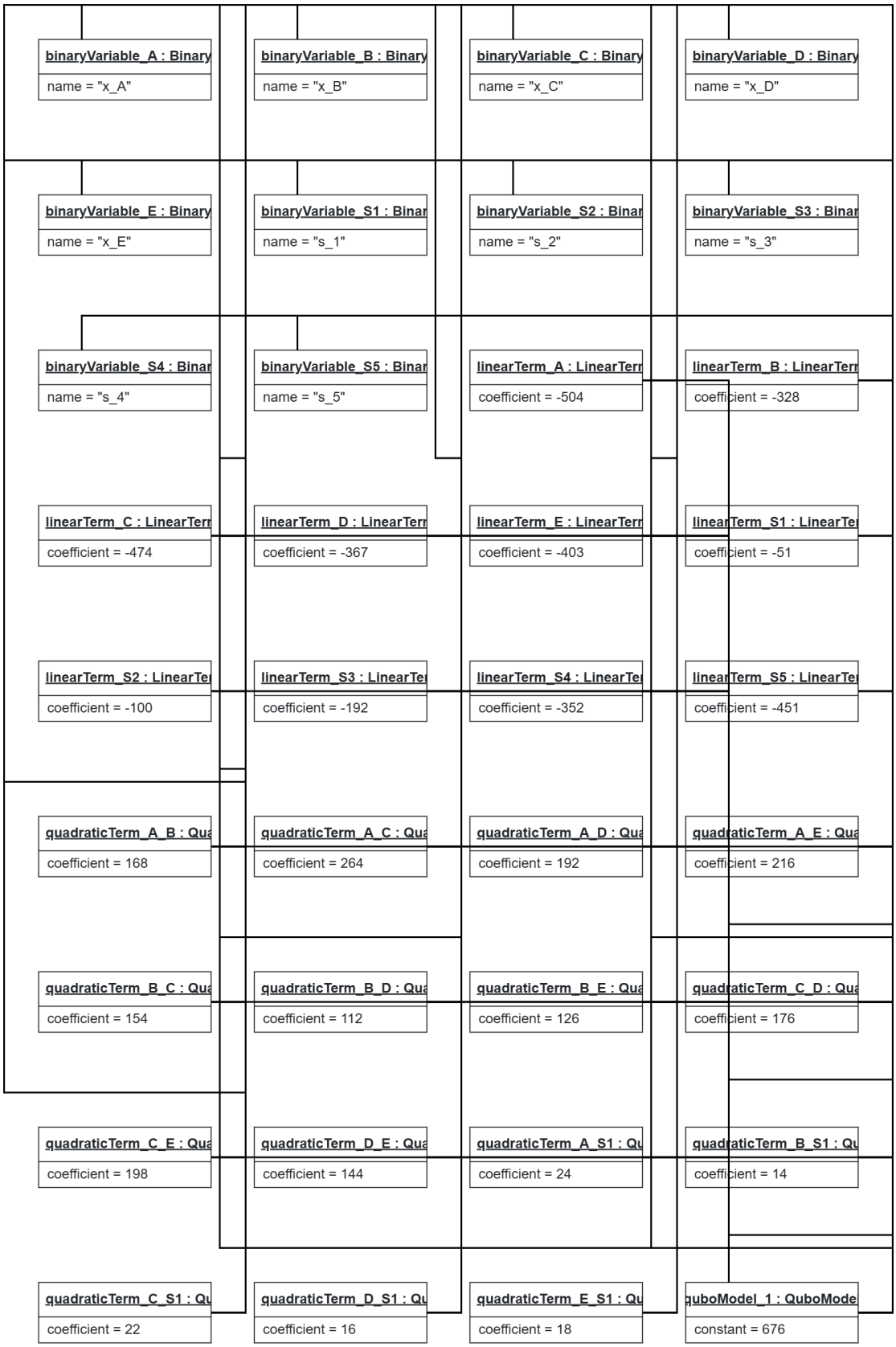


Figure 4: Solution DSL object diagram for Test Case 1.